



Lab 04: Arbitrary Axis Rotation

Mouse-Driven Axis Selection and World-Space Quaternion Rotation

Chi-Wei Yeh

yehchiwei.work@gmail.com

Computer Graphics

Abstract

Lab 04 extends the Lab 03 pipeline with arbitrary axis rotation. A three-state machine (Normal / PickAxis / AxisLocked) lets the user click a point in the viewport; the coordinate is unprojected onto the world-space $z = 0$ plane via the inverse clip matrix, defining a rotation axis through the origin. Rotation is accumulated with pre-multiplied quaternions, keeping the axis fixed in world space.

Keywords: arbitrary axis rotation, quaternion, ray casting, matrix unprojection

Latest Version: <https://coderscompass.org/articles/>

1. Overview

Lab 04 adds **arbitrary axis rotation** to the Lab 03 MVP pipeline. The user picks a rotation axis by clicking in the viewport; the click is unprojected to a world-space point that defines the axis through the origin, and rotation is accumulated via pre-multiplied quaternions. All Lab 03 controls are preserved.

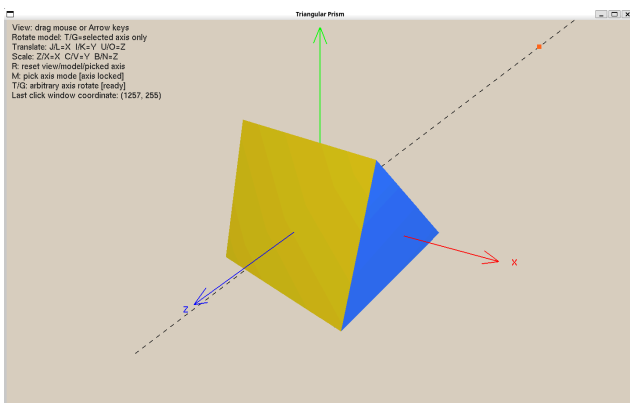


Figure 1: Lab 04 — dashed arbitrary axis and rotated triangular prism

From	Trigger	To
Normal	M	PickAxis
PickAxis	M	Normal
PickAxis	Left click (success)	AxisLocked
AxisLocked	M	PickAxis
Any	R	Normal

Table 1: State transitions

In **AxisLocked**, W/S/A/D/Q/E are suppressed; only T/G, translation, scaling, and camera orbit remain active.

2. State Machine & Controls

Interaction uses three explicit states so mouse behaviour is unambiguous.

Input	Action
M	Toggle PickAxis / resume from AxisLocked
Left click (Pick)	Set axis point → AxisLocked
T / G	Rotate ± along arbitrary axis
W/S A/D Q/E	Rotate local X / Y / Z
J/L I/K U/O	Translate X / Y / Z
Z/X C/V B/N	Scale X / Y / Z
Drag / Arrows	Orbit camera
R	Reset all → Normal

Table 2: Key bindings

3. Implementation Notes

3.1. Mouse-to-World Unprojection

Window coordinate (x, y) is converted to a world-space point in four steps:

1. NDC: $x_{\text{ndc}} = 2\frac{x}{W} - 1$, $y_{\text{ndc}} = 1 - 2\frac{y}{H}$.
2. Invert $M_{\text{clip}} = M_{\text{proj}} \cdot M_{\text{view}} \cdot M_{\text{orbit}}$ and unproject near/far clip points.
3. Ray-plane intersection with $z = 0$: $t = -p_{\text{near}} \cdot z / d \cdot z$, $p_{\text{world}} = p_{\text{near}} + td$.

Projecting onto $z = 0$ keeps the picked point, the visual axis, and the rotation formula all in the same coordinate frame with no depth buffer read required.

3.2. Quaternion Composition

Two distinct multiplication orders are used:

- **Local axis** (W/S/A/D/Q/E): $q = \text{norm}(q_{\text{orient}} \cdot Q_{\text{local}})$ — axis follows model pose.
- **World axis** (T/G): $q = \text{norm}(Q_{\text{axis}} \cdot q_{\text{orient}})$ — axis fixed in world space.

The pre-multiply rule for T/G is the central result of this lab.

4. Source Code

The uploaded source code is commented. In particular, comments are placed around the key setting, the three-state interaction flow, the mouse unprojection path, and the distinction between local-axis and

world-axis quaternion composition so the grading focus can be checked directly in code.

4.1. view_controller.hpp

A self-contained math library shared across all labs. It defines Vec3, Vec4, Matrix4 (with inverted() via cofactor expansion), Quaternion (Hamilton product, to_matrix(), from_axis_angle()), and the Transform, View, and Projection namespaces. OrbitViewController accumulates yaw/pitch from mouse drag and arrow keys and applies the resulting rotation to the OpenGL modelview stack. Vec4, Matrix4 * Vec4, and Matrix4::inverted() are the three additions over Lab 03, required by the unprojection pipeline.

cpp

```

1 #pragma once
2
3 #include <GL/freeglut.h>
4
5 #include <array>
6 #include <cassert>
7 #include <cmath>
8
9 namespace demo
10 {
11
12     static constexpr float kPi =
13         3.14159265358979323846f;
14     static constexpr float kEpsilon =
15         1e-6f;
16
17     inline float degrees_to_radians(float
18         degrees) {
19         return degrees * kPi / 180.0f;
20     }
21
22     struct Vec3
23     {
24         float x = 0.0f;
25         float y = 0.0f;
26         float z = 0.0f;
27
28         Vec3 operator+(const Vec3& rhs)
29         const {
30             return Vec3{x + rhs.x, y +
31                 rhs.y, z + rhs.z};
32         }
33
34         Vec3 operator-(const Vec3& rhs)
35         const {

```

```

30     return Vec3{x - rhs.x, y -
31     rhs.y, z - rhs.z};
32 }
33 Vec3 operator*(float scalar) const
34 {
35     return Vec3{x * scalar, y *
36     scalar, z * scalar};
37 }
38 struct Vec4
39 {
40     float x = 0.0f;
41     float y = 0.0f;
42     float z = 0.0f;
43     float w = 0.0f;
44 };
45
46 inline Vec3 cross(const Vec3& a, const
47 Vec3& b) {
48     return Vec3{a.y * b.z - a.z * b.y,
49     a.z * b.x - a.x * b.z,
50     a.x * b.y - a.y * b.x};
51 }
52 inline float dot(const Vec3& a, const
53 Vec3& b) {
54     return a.x * b.x + a.y * b.y + a.z
55     * b.z;
56 }
57 inline float length_squared(const Vec3&
58 v) {
59     return dot(v, v);
60 }
61 inline Vec3 normalize(const Vec3& v) {
62     const float squared_length =
63     length_squared(v);
64     assert(squared_length > kEpsilon &&
65     "normalize() requires a non-
66     zero vector");
67     const float length =
68     std::sqrt(squared_length);
69     return v * (1.0f / length);
70 }
71 struct Matrix4
72 {
73     std::array<float, 16> value{};
74
75     static Matrix4 identity() {
76         Matrix4 matrix{};
77         matrix.at(0, 0) = 1.0f;

```

```

78         matrix.at(1, 1) = 1.0f;
79         matrix.at(2, 2) = 1.0f;
80         matrix.at(3, 3) = 1.0f;
81         return matrix;
82     }
83
84     Matrix4 operator*(const Matrix4&
85     rhs) const {
86         Matrix4 result{};
87         for (int row = 0; row < 4; +
88         ++row) {
89             for (int col = 0; col < 4;
90             ++col) {
91                 float sum = 0.0f;
92                 for (int k = 0; k < 4;
93                 ++k) {
94                     sum += at(row, k) *
95                     rhs.at(k, col);
96                 }
97                 result.at(row, col) =
98                 sum;
99             }
100         }
101         return result;
102     }
103
104     Vec4 operator*(const Vec4& rhs)
105     const {
106         return Vec4{
107             at(0, 0) * rhs.x + at(0, 1)
108             * rhs.y + at(0, 2) * rhs.z +
109             at(0, 3) * rhs.w,
110             at(1, 0) * rhs.x + at(1, 1)
111             * rhs.y + at(1, 2) * rhs.z +
112             at(1, 3) * rhs.w,
113             at(2, 0) * rhs.x + at(2, 1)
114             * rhs.y + at(2, 2) * rhs.z +
115             at(2, 3) * rhs.w,
116             at(3, 0) * rhs.x + at(3, 1)
117             * rhs.y + at(3, 2) * rhs.z +
118             at(3, 3) * rhs.w,
119         };
120     }
121
122     Matrix4 inverted() const {
123         Matrix4 inverse{};
124         const float* m = value.data();
125         float inv[16];
126
127         inv[0] = m[5] * m[10] * m[15] -
128         m[5] * m[11] * m[14] -
129         m[9] * m[6] * m[15] +
130         m[9] * m[7] * m[14] +
131         m[13] * m[6] * m[11] -
132         m[13] * m[7] * m[10];

```

```

115
116     inv[4] = -m[4] * m[10] * m[15]
+ m[4] * m[11] * m[14] +
117     m[8] * m[6] * m[15] -
m[8] * m[7] * m[14] -
118     m[12] * m[6] * m[11] +
m[12] * m[7] * m[10];
119
120     inv[8] = m[4] * m[9] * m[15] -
m[4] * m[11] * m[13] -
121     m[8] * m[5] * m[15] +
m[8] * m[7] * m[13] +
122     m[12] * m[5] * m[11] -
m[12] * m[7] * m[9];
123
124     inv[12] = -m[4] * m[9] * m[14]
+ m[4] * m[10] * m[13] +
125     m[8] * m[5] * m[14] -
m[8] * m[6] * m[13] -
126     m[12] * m[5] * m[10]
+ m[12] * m[6] * m[9];
127
128     inv[1] = -m[1] * m[10] * m[15]
+ m[1] * m[11] * m[14] +
129     m[9] * m[2] * m[15] -
m[9] * m[3] * m[14] -
130     m[13] * m[2] * m[11] +
m[13] * m[3] * m[10];
131
132     inv[5] = m[0] * m[10] * m[15] -
m[0] * m[11] * m[14] -
133     m[8] * m[2] * m[15] +
m[8] * m[3] * m[14] +
134     m[12] * m[2] * m[11] -
m[12] * m[3] * m[10];
135
136     inv[9] = -m[0] * m[9] * m[15] +
m[0] * m[11] * m[13] +
137     m[8] * m[1] * m[15] -
m[8] * m[3] * m[13] -
138     m[12] * m[1] * m[11] +
m[12] * m[3] * m[9];
139
140     inv[13] = m[0] * m[9] * m[14] -
m[0] * m[10] * m[13] -
141     m[8] * m[1] * m[14] +
m[8] * m[2] * m[13] +
142     m[12] * m[1] * m[10]
- m[12] * m[2] * m[9];
143
144     inv[2] = m[1] * m[6] * m[15] -
m[1] * m[7] * m[14] -
145     m[5] * m[2] * m[15] +
m[5] * m[3] * m[14] +

```

```

146     m[13] * m[2] * m[7] -
m[13] * m[3] * m[6];
147
148     inv[6] = -m[0] * m[6] * m[15] +
m[0] * m[7] * m[14] +
149     m[4] * m[2] * m[15] -
m[4] * m[3] * m[14] -
150     m[12] * m[2] * m[7] +
m[12] * m[3] * m[6];
151
152     inv[10] = m[0] * m[5] * m[15] -
m[0] * m[7] * m[13] -
153     m[4] * m[1] * m[15] +
m[4] * m[3] * m[13] +
154     m[12] * m[1] * m[7] -
m[12] * m[3] * m[5];
155
156     inv[14] = -m[0] * m[5] * m[14]
+ m[0] * m[6] * m[13] +
157     m[4] * m[1] * m[14] -
m[4] * m[2] * m[13] -
158     m[12] * m[1] * m[6] +
m[12] * m[2] * m[5];
159
160     inv[3] = -m[1] * m[6] * m[11] +
m[1] * m[7] * m[10] +
161     m[5] * m[2] * m[11] -
m[5] * m[3] * m[10] -
162     m[9] * m[2] * m[7] +
m[9] * m[3] * m[6];
163
164     inv[7] = m[0] * m[6] * m[11] -
m[0] * m[7] * m[10] -
165     m[4] * m[2] * m[11] +
m[4] * m[3] * m[10] +
166     m[8] * m[2] * m[7] -
m[8] * m[3] * m[6];
167
168     inv[11] = -m[0] * m[5] * m[11]
+ m[0] * m[7] * m[9] +
169     m[4] * m[1] * m[11] -
m[4] * m[3] * m[9] -
170     m[8] * m[1] * m[7] +
m[8] * m[3] * m[5];
171
172     inv[15] = m[0] * m[5] * m[10] -
m[0] * m[6] * m[9] -
173     m[4] * m[1] * m[10] +
m[4] * m[2] * m[9] +
174     m[8] * m[1] * m[6] -
m[8] * m[2] * m[5];
175
176     float determinant =

```

```

177         m[0] * inv[0] + m[1] *
inv[4] + m[2] * inv[8] + m[3] *
inv[12];
178         assert(std::fabs(determinant) >
kEpsilon &&
179             "Matrix4::inverted()
requires an invertible matrix");
180
181         determinant = 1.0f /
determinant;
182         for (int i = 0; i < 16; ++i) {
183             inverse.value[i] = inv[i] *
determinant;
184         }
185         return inverse;
186     }
187
188     const float* data() const { return
value.data(); }
189     float at(int row, int col) const
{ return value[col * 4 + row]; }
190     float& at(int row, int col)
{ return value[col * 4 + row]; }
191 };
192
193 struct Quaternion
194 {
195     float w = 1.0f;
196     float x = 0.0f;
197     float y = 0.0f;
198     float z = 0.0f;
199
200     static Quaternion identity()
{ return Quaternion{}; }
201
202     static Quaternion
from_axis_angle(const Vec3& axis, float
degrees) {
203         assert(length_squared(axis) >
kEpsilon &&
204             "Quaternion::from_axis_angle() requires
a non-zero axis");
205         const Vec3 unit_axis =
normalize(axis);
206         const float radians = degrees *
kPi / 180.0f;
207         const float half_angle =
radians * 0.5f;
208         const float s =
std::sin(half_angle);
209         return
Quaternion{std::cos(half_angle),
unit_axis.x * s,
210             unit_axis.y *
s, unit_axis.z * s};

```

```

211     }
212
213     Quaternion operator*(const
Quaternion& rhs) const {
214         return Quaternion{
215             w * rhs.w - x * rhs.x - y *
rhs.y - z * rhs.z,
216             w * rhs.x + x * rhs.w + y *
rhs.z - z * rhs.y,
217             w * rhs.y - x * rhs.z + y *
rhs.w + z * rhs.x,
218             w * rhs.z + x * rhs.y - y *
rhs.x + z * rhs.w,
219             };
220     }
221
222     Quaternion normalized() const {
223         const float squared_length = w
* w + x * x + y * y + z * z;
224         assert(squared_length >
kEpsilon &&
225             "Quaternion::normalized() requires a
non-zero quaternion");
226         const float length =
std::sqrt(squared_length);
227         return Quaternion{w / length,
x / length, y / length, z / length};
228     }
229
230     Quaternion conjugate() const
{ return Quaternion{w, -x, -y, -z}; }
231
232     Vec3 rotate(const Vec3& v) const {
233         const Quaternion qv{0.0f, v.x,
v.y, v.z};
234         const Quaternion result =
(*this) * qv * conjugate();
235         return Vec3{result.x, result.y,
result.z};
236     }
237
238     Matrix4 to_matrix() const {
239         // Assumes this quaternion is
already normalized by the caller.
240         const float xx = x * x;
241         const float yy = y * y;
242         const float zz = z * z;
243         const float xy = x * y;
244         const float xz = x * z;
245         const float yz = y * z;
246         const float wx = w * x;
247         const float wy = w * y;
248         const float wz = w * z;
249

```

```

250     Matrix4 matrix =
Matrix4::identity();
251     matrix.at(0, 0) = 1.0f - 2.0f *
(yy + zz);
252     matrix.at(0, 1) = 2.0f * (xy -
wz);
253     matrix.at(0, 2) = 2.0f * (xz +
wy);
254     matrix.at(1, 0) = 2.0f * (xy +
wz);
255     matrix.at(1, 1) = 1.0f - 2.0f *
(xx + zz);
256     matrix.at(1, 2) = 2.0f * (yz -
wx);
257     matrix.at(2, 0) = 2.0f * (xz -
wy);
258     matrix.at(2, 1) = 2.0f * (yz +
wx);
259     matrix.at(2, 2) = 1.0f - 2.0f *
(xx + yy);
260     return matrix;
261 }
262 };
263
264 namespace Transform
265 {
266
267 inline Matrix4 rotation_x(float
degrees) {
268     const float radians =
degrees_to_radians(degrees);
269     const float c = std::cos(radians);
270     const float s = std::sin(radians);
271     Matrix4 matrix =
Matrix4::identity();
272     matrix.at(1, 1) = c;
273     matrix.at(1, 2) = -s;
274     matrix.at(2, 1) = s;
275     matrix.at(2, 2) = c;
276     return matrix;
277 }
278
279 inline Matrix4 rotation_y(float
degrees) {
280     const float radians =
degrees_to_radians(degrees);
281     const float c = std::cos(radians);
282     const float s = std::sin(radians);
283     Matrix4 matrix =
Matrix4::identity();
284     matrix.at(0, 0) = c;
285     matrix.at(0, 2) = s;
286     matrix.at(2, 0) = -s;
287     matrix.at(2, 2) = c;
288     return matrix;

```

```

289 }
290
291 inline Matrix4 rotation_z(float
degrees) {
292     const float radians =
degrees_to_radians(degrees);
293     const float c = std::cos(radians);
294     const float s = std::sin(radians);
295     Matrix4 matrix =
Matrix4::identity();
296     matrix.at(0, 0) = c;
297     matrix.at(0, 1) = -s;
298     matrix.at(1, 0) = s;
299     matrix.at(1, 1) = c;
300     return matrix;
301 }
302
303 inline Matrix4 translation(float tx,
float ty, float tz) {
304     Matrix4 matrix =
Matrix4::identity();
305     matrix.at(0, 3) = tx;
306     matrix.at(1, 3) = ty;
307     matrix.at(2, 3) = tz;
308     return matrix;
309 }
310
311 inline Matrix4 translation(const Vec3&
offset) {
312     return
Transform::translation(offset.x,
offset.y, offset.z);
313 }
314
315 inline Matrix4 scaling(float sx, float
sy, float sz) {
316     Matrix4 matrix{};
317     matrix.at(0, 0) = sx;
318     matrix.at(1, 1) = sy;
319     matrix.at(2, 2) = sz;
320     matrix.at(3, 3) = 1.0f;
321     return matrix;
322 }
323
324 inline Matrix4 scaling(const Vec3&
scale) {
325     return Transform::scaling(scale.x,
scale.y, scale.z);
326 }
327
328 inline Matrix4 trs(const Vec3&
position, const Quaternion& rotation,
const Vec3& scale) {
329     return translation(position) *
rotation.to_matrix() * scaling(scale);
330 }

```

```

331 }
332
333 } // namespace Transform
334
335 namespace View
336 {
337
338 inline Matrix4 look_at(const Vec3& eye,
339   const Vec3& center, const Vec3& up) {
340   const Vec3 view_direction = center
341     - eye;
342   assert(length_squared(view_direction) >
343     kEpsilon &&
344     "View::look_at() requires
345     eye and center to be different");
346   const Vec3 forward =
347     normalize(center - eye);
348   const Vec3 side_unnormalized =
349     cross(forward, up);
350   assert(
351     length_squared(side_unnormalized) >
352     kEpsilon &&
353     "View::look_at() requires up to
354     be non-collinear with view direction");
355   const Vec3 side =
356     normalize(side_unnormalized);
357   const Vec3 camera_up = cross(side,
358     forward);
359   Matrix4 matrix =
360     Matrix4::identity();
361   matrix.at(0, 0) = side.x;
362   matrix.at(0, 1) = side.y;
363   matrix.at(0, 2) = side.z;
364   matrix.at(0, 3) = -dot(side, eye);
365   matrix.at(1, 0) = camera_up.x;
366   matrix.at(1, 1) = camera_up.y;
367   matrix.at(1, 2) = camera_up.z;
368   matrix.at(1, 3) = -dot(camera_up,
369     eye);
370   matrix.at(2, 0) = -forward.x;
371   matrix.at(2, 1) = -forward.y;
372   matrix.at(2, 2) = -forward.z;
373   matrix.at(2, 3) = dot(forward,
374     eye);
375   return matrix;
376 }
377 } // namespace View
378
379 namespace Projection
380 {

```

```

371 inline Matrix4 orthographic(float left,
372   float right, float bottom, float top,
373   float
374   near_plane, float far_plane) {
375   assert(std::fabs(right - left) >
376     kEpsilon &&
377     "Projection::orthographic()
378     requires left != right");
379   assert(std::fabs(top - bottom) >
380     kEpsilon &&
381     "Projection::orthographic()
382     requires bottom != top");
383   assert(std::fabs(far_plane -
384     near_plane) > kEpsilon &&
385     "Projection::orthographic()
386     requires near != far");
387   Matrix4 matrix{};
388   matrix.at(0, 0) = 2.0f / (right -
389     left);
390   matrix.at(1, 1) = 2.0f / (top -
391     bottom);
392   matrix.at(2, 2) = -2.0f /
393     (far_plane - near_plane);
394   matrix.at(0, 3) = -(right + left) /
395     (right - left);
396   matrix.at(1, 3) = -(top + bottom) /
397     (top - bottom);
398   matrix.at(2, 3) = -(far_plane +
399     near_plane) / (far_plane - near_plane);
400   matrix.at(3, 3) = 1.0f;
401   return matrix;
402 }
403
404 inline Matrix4 perspective(float
405   fov_y_degrees, float aspect, float
406   near_plane,
407   float
408   far_plane) {
409   assert(fov_y_degrees > 0.0f &&
410     fov_y_degrees < 180.0f &&
411     "Projection::perspective()
412     requires 0 < fov_y_degrees < 180");
413   assert(aspect > kEpsilon &&
414     "Projection::perspective()
415     requires a positive aspect ratio");
416   assert(near_plane > 0.0f &&
417     "Projection::perspective()
418     requires near_plane > 0");
419   assert(far_plane > near_plane &&
420     "Projection::perspective()
421     requires far_plane > near_plane");
422   const float radians =
423     degrees_to_radians(fov_y_degrees);
424   const float f = 1.0f /
425     std::tan(radians * 0.5f);

```

```

402     Matrix4 matrix{};
403     matrix.at(0, 0) = f / aspect;
404     matrix.at(1, 1) = f;
405     matrix.at(2, 2) = (far_plane +
near_plane) / (near_plane - far_plane);
406     matrix.at(2, 3) =
407         (2.0f * far_plane *
near_plane) / (near_plane - far_plane);
408     matrix.at(3, 2) = -1.0f;
409     return matrix;
410 }
411
412 } // namespace Projection
413
414 // Orbit camera that rotates the view
around the world origin.
415 // Call apply_rotation() each frame
before drawing to apply the accumulated
416 // yaw/pitch. Input is accepted via
mouse drag (left button) and arrow
keys.
417 struct OrbitViewController
418 {
419     static constexpr float kInitialYaw
= 25.0f;
420     static constexpr float
kInitialPitch = 0.0f;
421
422     float yaw = kInitialYaw;
423     float pitch = kInitialPitch;
424
425     bool dragging = false;
426     int last_x = 0;
427     int last_y = 0;
428
429     float mouse_sensitivity = 0.4f;
430     float key_step = 5.0f;
431
432     void apply_rotation() const {
433         // This is an orbit-style post-
view rotation: the base camera is first
434         // built by View::look_at(),
then yaw/pitch rotate the already-built
435         // view frame instead of
recomputing eye/center from spherical
angles.
436         const Matrix4 rotation =
437             Transform::rotation_x(pitch) *
Transform::rotation_y(yaw);
438         glMultMatrixf(rotation.data());
439     }
440
441     void reset() {
442         yaw = kInitialYaw;
443         pitch = kInitialPitch;

```

```

444         dragging = false;
445         glutPostRedisplay();
446     }
447
448     void on_mouse_button(int button,
int state, int x, int y) {
449         if (button != GLUT_LEFT_BUTTON)
return;
450
451         dragging = (state ==
GLUT_DOWN);
452         last_x = x;
453         last_y = y;
454     }
455
456     void on_mouse_drag(int x, int y) {
457         if (!dragging) return;
458
459         rotate_by((x - last_x) *
mouse_sensitivity,
460                 (y - last_y) *
mouse_sensitivity);
461         last_x = x;
462         last_y = y;
463     }
464
465     void on_special_key(int key) {
466         switch (key) {
467             case GLUT_KEY_LEFT:
rotate_by(-key_step,
468         0.0f);
469             break;
470             case GLUT_KEY_RIGHT:
rotate_by(key_step,
471         0.0f);
472             break;
473             case GLUT_KEY_UP:
rotate_by(0.0f, -
474         key_step);
475             break;
476             case GLUT_KEY_DOWN:
rotate_by(0.0f,
477         key_step);
478             break;
479             default:
break;
480         }
481     }
482 }
483
484 private:
485     void rotate_by(float delta_yaw,
float delta_pitch) {
486         yaw += delta_yaw;
487         pitch += delta_pitch;
488

```

```

489         if (pitch > 89.0f) pitch =
89.0f;
490         if (pitch < -89.0f) pitch =
-89.0f;
491
492         glutPostRedisplay();
493     }
494 };
495
496 } // namespace demo
497

```

4.2. triangular_prism.cpp

The main application file. It wires all GLUT callbacks through a single App struct stored as GLUT window data, keeping all mutable state in one place.

ModelTransform owns orientation (quaternion), position, and scale. `on_keyboard()` dispatches all transform keys and receives a `const Vec3*` for the arbitrary axis — `nullptr` in `Normal/PickAxis`, non-null in `Axis-Locked` — so T/G silently no-op when no axis is selected.

App owns the view controller, model transform, current Mode enum, and `selected_point`. `selected_axis()` returns `&selected_point` only in `AxisLocked`, making axis availability a single-source-of-truth check.

mouse_cb branches on mode: in `PickAxis` it calls `unproject_mouse_to_world_plane`, transitions to `Axis-Locked` on success, and prints the window coordinate to `stdout`; otherwise it forwards the event to `OrbitViewController` for camera drag.

display_cb draws in three layers: (1) world-space geometry (coordinate axes and the optional dashed axis line) before `model.apply()`; (2) model geometry inside `glPushMatrix/glPopMatrix`; (3) the 2D controls overlay under an orthographic projection with depth test disabled.

cpp

```

1 #include <GL/freeglut.h>
2
3 #include <cctype>
4 #include <cstdio>
5
6 #include "view_controller.hpp"
7
8 namespace demo
9 {
10

```

```

11 static void draw_axis_label(float x,
float y, float z, const char label) {
12     glRasterPos3f(x, y, z);
13     glutBitmapCharacter(GLUT_BITMAP_HELVETIC
label);
14 }
15
16 static void draw_axes() {
17     constexpr float axis_length = 2.0f;
18     constexpr float arrow_size = 0.15f;
19
20     glPushAttrib(GL_ENABLE_BIT |
GL_LINE_BIT);
21     glDisable(GL_LIGHTING);
22     glLineWidth(2.0f);
23
24     glBegin(GL_LINES);
25     glColor3f(1.0f, 0.0f, 0.0f);
26     glVertex3f(0.0f, 0.0f, 0.0f);
27     glVertex3f(axis_length, 0.0f,
0.0f);
28     glVertex3f(axis_length, 0.0f,
0.0f);
29     glVertex3f(axis_length -
arrow_size, arrow_size * 0.5f, 0.0f);
30     glVertex3f(axis_length, 0.0f,
0.0f);
31     glVertex3f(axis_length -
arrow_size, -arrow_size * 0.5f, 0.0f);
32
33     glColor3f(0.0f, 1.0f, 0.0f);
34     glVertex3f(0.0f, 0.0f, 0.0f);
35     glVertex3f(0.0f, axis_length,
0.0f);
36     glVertex3f(0.0f, axis_length,
0.0f);
37     glVertex3f(arrow_size * 0.5f,
axis_length - arrow_size, 0.0f);
38     glVertex3f(0.0f, axis_length,
0.0f);
39     glVertex3f(-arrow_size * 0.5f,
axis_length - arrow_size, 0.0f);
40
41     glColor3f(0.0f, 0.0f, 1.0f);
42     glVertex3f(0.0f, 0.0f, 0.0f);
43     glVertex3f(0.0f, 0.0f,
axis_length);
44     glVertex3f(0.0f, 0.0f,
axis_length);
45     glVertex3f(0.0f, arrow_size * 0.5f,
axis_length - arrow_size);
46     glVertex3f(0.0f, 0.0f,
axis_length);
47     glVertex3f(0.0f, -arrow_size *
0.5f, axis_length - arrow_size);

```

```

48     glEnd();
49
50     glColor3f(1.0f, 0.0f, 0.0f);
51     draw_axis_label(axis_length +
52         0.12f, 0.0f, 0.0f, 'X');
53     glColor3f(0.0f, 1.0f, 0.0f);
54     draw_axis_label(0.0f, axis_length +
55         0.12f, 0.0f, 'Y');
56     glColor3f(0.0f, 0.0f, 1.0f);
57     draw_axis_label(0.0f, 0.0f,
58         axis_length + 0.12f, 'Z');
59
60     glPopAttrib();
61 }
62
63 static void draw_bitmap_text(int x, int
64     y, const char* text) {
65     glRasterPos2i(x, y);
66     for (const char* p = text; *p !=
67         '\0'; ++p) {
68         glutBitmapCharacter(GLUT_BITMAP_HELVETIC
69             *p);
70     }
71 }
72
73 static Matrix4
74     build_scene_projection_matrix(int
75         width, int height) {
76     return
77         Projection::perspective(45.0f,
78             static_cast<float>(width) / height,
79             0.1f, 100.0f);
80 }
81
82 static Matrix4
83     build_scene_view_matrix() {
84     return View::look_at(Vec3{0.0f,
85         2.0f, 5.0f}, Vec3{0.0f, 0.0f, 0.0f},
86         Vec3{0.0f,
87             1.0f, 0.0f});
88 }
89
90 static Matrix4
91     build_scene_orbit_matrix(const
92         OrbitViewController& view) {
93     return
94         Transform::rotation_x(view.pitch) *
95         Transform::rotation_y(view.yaw);
96 }
97
98 static Vec3 homogenize_point(const
99     Vec4& value) {
100     assert(std::fabs(value.w) >
101         kEpsilon &&

```

```

83         "homogenize_point() requires
84         a non-zero w");
85     return Vec3{value.x / value.w,
86         value.y / value.w, value.z / value.w};
87 }
88
89 static bool
90     unproject_mouse_to_world_plane(
91         int mouse_x, int mouse_y, int
92         width, int height,
93         const OrbitViewController& view,
94         Vec3* world_point) {
95     if (height <= 0) return false;
96
97     const float ndc_x = 2.0f *
98         static_cast<float>(mouse_x) / width -
99         1.0f;
100     const float ndc_y =
101         1.0f - 2.0f *
102         static_cast<float>(mouse_y) /
103         static_cast<float>(height);
104
105     const Matrix4 inverse_clip_to_world
106         =
107         (build_scene_projection_matrix(width,
108             height) * build_scene_view_matrix() *
109             build_scene_orbit_matrix(view))
110         .inverted();
111
112     const Vec3 near_point =
113         homogenize_point(
114             inverse_clip_to_world *
115             Vec4{ndc_x, ndc_y, -1.0f, 1.0f});
116     const Vec3 far_point =
117         homogenize_point(
118             inverse_clip_to_world *
119             Vec4{ndc_x, ndc_y, 1.0f, 1.0f});
120     const Vec3 ray_direction =
121         far_point - near_point;
122
123     if (std::fabs(ray_direction.z) <=
124         kEpsilon) return false;
125
126     const float t = -near_point.z /
127         ray_direction.z;
128     if (t < 0.0f) return false;
129
130     *world_point = near_point +
131         ray_direction * t;
132     return true;
133 }
134
135 static void draw_selected_axis(const
136     Vec3& selected_point) {

```

```

117     const Vec3 ray_direction =
normalize(selected_point);
118     const float ray_length = 6.0f;
119     const Vec3 ray_start =
ray_direction * -ray_length;
120     const Vec3 ray_end = ray_direction
* ray_length;
121
122     glPushAttrib(GL_ENABLE_BIT |
GL_LINE_BIT | GL_POINT_BIT);
123     glDisable(GL_LIGHTING);
124     glLineWidth(2.0f);
125     glEnable(GL_LINE_STIPPLE);
126     glLineStipple(1, 0x00FF);
127     glColor3f(0.1f, 0.1f, 0.1f);
128
129     glBegin(GL_LINES);
130     glVertex3f(ray_start.x,
ray_start.y, ray_start.z);
131     glVertex3f(ray_end.x, ray_end.y,
ray_end.z);
132     glEnd();
133
134     glDisable(GL_LINE_STIPPLE);
135     glPointSize(10.0f);
136     glColor3f(1.0f, 0.4f, 0.1f);
137     glBegin(GL_POINTS);
138     glVertex3f(selected_point.x,
selected_point.y, selected_point.z);
139     glEnd();
140     glPopAttrib();
141 }
142
143 static void draw_controls_overlay(bool
pick_mode_armed, bool
has_selected_axis,
144                                     int
last_click_x, int last_click_y) {
145     const int width =
glutGet(GLUT_WINDOW_WIDTH);
146     const int height =
glutGet(GLUT_WINDOW_HEIGHT);
147     char pick_line[96];
148     char axis_line[96];
149     const char* pick_mode_text = "off";
150     if (pick_mode_armed) {
151         pick_mode_text = "waiting for
click";
152     } else if (has_selected_axis) {
153         pick_mode_text = "axis locked";
154     }
155     std::snprintf(pick_line,
sizeof(pick_line), "M: pick axis mode
[%s]",
156                                     pick_mode_text);

```

```

157     std::snprintf(axis_line,
sizeof(axis_line), "T/G: arbitrary axis
rotate [%s]",
158                                     has_selected_axis ?
"ready" : "need picked point");
159     const char* rotate_line =
has_selected_axis
160                                     ?
"Rotate model: T/G=selected axis only"
161                                     :
"Rotate model: W/S=X A/D=Y Q/E=Z";
162     constexpr const char*
kOverlayLinesPrefix[] = {
163         "View: drag mouse or Arrow
keys",
164     };
165     constexpr const char*
kOverlayLinesSuffix[] = {
166         "Translate: J/L=X I/K=Y U/
0=Z",
167         "Scale: Z/X=X C/V=Y B/N=Z",
168         "R: reset view/model/picked
axis",
169     };
170
171     glPushAttrib(GL_ENABLE_BIT);
172
173     glMatrixMode(GL_PROJECTION);
174     glPushMatrix();
175     glLoadIdentity();
176     const Matrix4 overlay_projection =
Projection::orthographic(0.0f,
static_cast<float>(width), 0.0f,
178 static_cast<float>(height), -1.0f,
1.0f);
179     glMultMatrixf(overlay_projection.data())
180
181     glMatrixMode(GL_MODELVIEW);
182     glPushMatrix();
183     glLoadIdentity();
184
185     glDisable(GL_LIGHTING);
186     glDisable(GL_DEPTH_TEST);
187     glColor3f(0.15f, 0.15f, 0.15f);
188
189     int line_index = 0;
190     for (const char* line :
kOverlayLinesPrefix) {
191         draw_bitmap_text(12, height -
24 - line_index * 22, line);
192         ++line_index;
193     }
194     draw_bitmap_text(12, height - 24 -
line_index * 22, rotate_line);

```

```

195     ++line_index;
196     for (const char* line :
197         kOverlayLinesSuffix) {
198         draw_bitmap_text(12, height -
199             24 - line_index * 22, line);
200         ++line_index;
201     }
202     draw_bitmap_text(12, height - 24 -
203         line_index * 22,
204         pick_line);
205     ++line_index;
206     if (has_selected_axis) {
207         char click_line[96];
208         std::snprintf(click_line,
209             sizeof(click_line),
210             "Last click
211             window coordinate: (%d, %d)",
212             last_click_x,
213             last_click_y);
214         draw_bitmap_text(12, height -
215             24 - line_index * 22, click_line);
216     }
217     glPopMatrix();
218     glMatrixMode(GL_PROJECTION);
219     glPopMatrix();
220     glMatrixMode(GL_MODELVIEW);
221     glPopAttrib();
222 }
223
224 struct ModelTransform
225 {
226     static constexpr float
227     kRotationStep = 5.0f;
228     static constexpr float
229     kTranslateStep = 0.1f;
230     static constexpr float kScaleStep =
231     0.1f;
232     static constexpr float kMinScale =
233     0.1f;
234     Quaternion orientation =
235     Quaternion::identity();
236     Vec3 position{};
237     Vec3 scale{1.0f, 1.0f, 1.0f};
238     void apply() const {

```

```

235         const Matrix4 transform =
236         Transform::trs(position, orientation,
237             scale);
238         glmMultMatrixf(transform.data());
239     }
240     void reset() {
241         orientation =
242         Quaternion::identity();
243         position = Vec3{};
244         scale = Vec3{1.0f, 1.0f, 1.0f};
245         glutPostRedisplay();
246     }
247     // Key setting:
248     // W/S, A/D, Q/E rotate around the
249     // object's local X/Y/Z axes.
250     // T/G rotate around the arbitrary
251     // axis selected by the latest mouse
252     // click.
253     // J/L, I/K, U/O translate the
254     // model. Z/X, C/V, B/N scale the model.
255     bool on_keyboard(unsigned char key,
256         const Vec3* arbitrary_axis,
257         bool
258         arbitrary_axis_locked) {
259         key = static_cast<unsigned
260         char>(std::tolower(key));
261         switch (key) {
262             case 'w':
263                 if
264                 (arbitrary_axis_locked) return false;
265                 rotate_about_local_axis(Vec3{1.0f,
266                     0.0f, 0.0f}, kRotationStep);
267                 break;
268             case 's':
269                 if
270                 (arbitrary_axis_locked) return false;
271                 rotate_about_local_axis(Vec3{1.0f,
272                     0.0f, 0.0f}, -kRotationStep);
273                 break;
274             case 'a':
275                 if
276                 (arbitrary_axis_locked) return false;
277                 rotate_about_local_axis(Vec3{0.0f,
278                     1.0f, 0.0f}, -kRotationStep);
279                 break;
280             case 'd':
281                 if
282                 (arbitrary_axis_locked) return false;
283                 rotate_about_local_axis(Vec3{0.0f,
284                     1.0f, 0.0f}, kRotationStep);
285                 break;

```

```

270         case 'q':
271             if
(arbitrary_axis_locked) return false;
272 rotate_about_local_axis(Vec3{0.0f,
0.0f, 1.0f}, -kRotationStep);
273         break;
274         case 'e':
275             if
(arbitrary_axis_locked) return false;
276 rotate_about_local_axis(Vec3{0.0f,
0.0f, 1.0f}, kRotationStep);
277         break;
278         case 't':
279             if (arbitrary_axis ==
nullptr) return false;
280 rotate_about_world_axis(*arbitrary_axis,
kRotationStep);
281         break;
282         case 'g':
283             if (arbitrary_axis ==
nullptr) return false;
284 rotate_about_world_axis(*arbitrary_axis,
-kRotationStep);
285         break;
286         case 'j':
287             position.x -=
kTranslateStep;
288         break;
289         case 'l':
290             position.x +=
kTranslateStep;
291         break;
292         case 'i':
293             position.y +=
kTranslateStep;
294         break;
295         case 'k':
296             position.y -=
kTranslateStep;
297         break;
298         case 'u':
299             position.z +=
kTranslateStep;
300         break;
301         case 'o':
302             position.z -=
kTranslateStep;
303         break;
304         case 'z':
305             scale.x =
clamp_scale(scale.x - kScaleStep);
306         break;
307         case 'x':

```

```

308             scale.x += kScaleStep;
309         break;
310         case 'c':
311             scale.y =
clamp_scale(scale.y - kScaleStep);
312         break;
313         case 'v':
314             scale.y += kScaleStep;
315         break;
316         case 'b':
317             scale.z =
clamp_scale(scale.z - kScaleStep);
318         break;
319         case 'n':
320             scale.z += kScaleStep;
321         break;
322         default:
323             return false;
324     }
325     glutPostRedisplay();
326     return true;
327 }
328
329 private:
330     static float clamp_scale(float
value) {
331         return value < kMinScale ?
kMinScale : value;
332     }
333
334     void rotate_about_local_axis(const
Vec3& axis, float degrees) {
335         orientation =
(orientation *
Quaternion::from_axis_angle(axis,
degrees))
336         .normalized();
337     }
338 }
339
340     void rotate_about_world_axis(const
Vec3& axis, float degrees) {
341         orientation =
(Quaternion::from_axis_angle(axis,
degrees) * orientation)
342         .normalized();
343     }
344 }
345 };
346
347 static void draw_triangular_prism() {
348     const float yb = -1.0f, yt = 1.0f;
349
350     const float ax = 0.000f, az =
1.000f;
351     const float bx = -0.866f, bz =
-0.500f;

```

```

352     const float cx = 0.866f, cz =
      -0.500f;
353
354     glBegin(GL_TRIANGLES);
355
356     glColor3f(0.85f, 0.20f, 0.20f);
357     glNormal3f(0.0f, -1.0f, 0.0f);
358     glVertex3f(ax, yb, az);
359     glVertex3f(cx, yb, cz);
360     glVertex3f(bx, yb, bz);
361
362     glColor3f(0.20f, 0.45f, 1.00f);
363     glNormal3f(0.0f, 1.0f, 0.0f);
364     glVertex3f(ax, yt, az);
365     glVertex3f(bx, yt, bz);
366     glVertex3f(cx, yt, cz);
367
368     glColor3f(0.20f, 0.75f, 0.30f);
369     glNormal3f(-0.866f, 0.0f, 0.500f);
370     glVertex3f(ax, yb, az);
371     glVertex3f(ax, yt, az);
372     glVertex3f(bx, yt, bz);
373     glVertex3f(ax, yb, az);
374     glVertex3f(bx, yt, bz);
375     glVertex3f(bx, yb, bz);
376
377     glColor3f(0.90f, 0.80f, 0.10f);
378     glNormal3f(0.0f, 0.0f, -1.0f);
379     glVertex3f(bx, yb, bz);
380     glVertex3f(bx, yt, bz);
381     glVertex3f(cx, yt, cz);
382     glVertex3f(bx, yb, bz);
383     glVertex3f(cx, yt, cz);
384     glVertex3f(cx, yb, cz);
385
386     glColor3f(0.70f, 0.20f, 0.90f);
387     glNormal3f(0.866f, 0.0f, 0.500f);
388     glVertex3f(cx, yb, cz);
389     glVertex3f(cx, yt, cz);
390     glVertex3f(ax, yt, az);
391     glVertex3f(cx, yb, cz);
392     glVertex3f(ax, yt, az);
393     glVertex3f(ax, yb, az);
394
395     glEnd();
396 }
397
398 struct App final
399 {
400     enum class Mode
401     {
402         kNormal,
403         kPickAxis,
404         kAxisLocked,
405     };

```

```

406
407     OrbitViewController view{};
408     ModelTransform model{};
409     Mode mode = Mode::kNormal;
410     int last_click_x = 0;
411     int last_click_y = 0;
412     Vec3 selected_point{};
413
414     const Vec3* selected_axis() const {
415         return mode ==
416             Mode::kAxisLocked ? &selected_point :
417             nullptr;
418     }
419
420     static void reshape_cb(int w, int
421         h) {
422         if (h == 0) h = 1;
423
424         glViewport(0, 0, w, h);
425         glMatrixMode(GL_PROJECTION);
426         glLoadIdentity();
427         const Matrix4 projection =
428             build_scene_projection_matrix(w, h);
429         glMultMatrixf(projection.data());
430         glMatrixMode(GL_MODELVIEW);
431     }
432
433     static void display_cb() {
434         auto* self =
435             static_cast<App*>(glutGetWindowData());
436
437         glClear(GL_COLOR_BUFFER_BIT |
438             GL_DEPTH_BUFFER_BIT);
439         glLoadIdentity();
440         const Matrix4 view_matrix =
441             build_scene_view_matrix();
442         glMultMatrixf(view_matrix.data());
443
444         self->view.apply_rotation();
445
446         draw_axes();
447         if (self->mode ==
448             Mode::kAxisLocked) {
449             draw_selected_axis(self-
450                 >selected_point);
451         }
452         glPushMatrix();
453         self->model.apply();
454         draw_triangular_prism();
455         glPopMatrix();
456         draw_controls_overlay(self-
457             >mode == Mode::kPickAxis,
458             self-
459             >mode == Mode::kAxisLocked,

```

```

449         self->last_click_x, self->last_click_y);
450         glutSwapBuffers();
451     }
452
453     static void mouse_cb(int button,
454 int state, int x, int y) {
455         auto* self =
456 static_cast<App*>(glutGetWindowData());
457         if (button == GLUT_LEFT_BUTTON
458 && state == GLUT_DOWN &&
459 self->mode ==
460 Mode::kPickAxis) {
461             Vec3 world_point{};
462             const int width =
463 glutGet(GLUT_WINDOW_WIDTH);
464             const int height =
465 glutGet(GLUT_WINDOW_HEIGHT);
466             if
467 (unproject_mouse_to_world_plane(x, y,
468 width, height, self->view,
469 &world_point)) {
470                 if
471 (length_squared(world_point) <=
472 kEpsilon) {
473                     self->mode =
474 Mode::kNormal;
475                     glutPostRedisplay();
476                     return;
477                 }
478                 self->selected_point =
479 world_point;
480                 self->mode =
481 Mode::kAxisLocked;
482                 self->last_click_x = x;
483                 self->last_click_y = y;
484                 std::printf("Mouse
485 click window coordinate: (%d, %d)\n",
486 x, y);
487             }
488             glutPostRedisplay();
489             return;
490         }
491         self->view.on_mouse_button(button, state, x,
492 y);
493     }
494
495     static void motion_cb(int x, int y)
496 {
497         auto* self =
498 static_cast<App*>(glutGetWindowData());
499         self->view.on_mouse_drag(x, y);
500     }

```

```

483
484     static void keyboard_cb(unsigned
485 char key, int, int) {
486         auto* self =
487 static_cast<App*>(glutGetWindowData());
488         if (key == 'r' || key == 'R') {
489             self->view.reset();
490             self->model.reset();
491             self->mode = Mode::kNormal;
492             return;
493         }
494         if (key == 'm' || key == 'M') {
495             self->mode =
496 Mode::kPickAxis ? Mode::kNormal :
497 Mode::kPickAxis;
498             glutPostRedisplay();
499             return;
500         }
501         self->model.on_keyboard(key,
502 self->selected_axis(),
503 self->mode == Mode::kAxisLocked);
504     }
505
506     static void special_cb(int key,
507 int, int) {
508         auto* self =
509 static_cast<App*>(glutGetWindowData());
510         self->view.on_special_key(key);
511     }
512 };
513
514 // namespace demo
515
516 int main(int argc, char** argv) {
517     glutInit(&argc, argv);
518     glutInitDisplayMode(GLUT_DOUBLE |
519 GLUT_RGB | GLUT_DEPTH);
520     glutInitWindowSize(500, 500);
521     glutInitWindowPosition(600, 80);
522     const int win =
523 glutCreateWindow("Triangular Prism");
524     if (win <= 0) return 1;
525
526     glEnable(GL_DEPTH_TEST);
527     glEnable(GL_LIGHTING);
528     glEnable(GL_LIGHT0);
529     glEnable(GL_COLOR_MATERIAL);
530
531     const float lightPos[] = {2.f, 4.f,
532 5.f, 1.f};
533     const float ambient[] = {0.3f,
534 0.3f, 0.3f, 1.f};

```

```
526     const float diffuse[] = {0.8f,
527     0.8f, 0.8f, 1.f};
528     glLightfv(GL_LIGHT0, GL_AMBIENT,
529     ambient);
530     glLightfv(GL_LIGHT0, GL_DIFFUSE,
531     diffuse);
532     glLightfv(GL_LIGHT0, GL_POSITION,
533     lightPos);
534     glClearColor(0.85f, 0.82f, 0.75f,
535     1.f);
536
537     static demo::App app{};
538     glutSetWindowData(&app);
539
540     glutReshapeFunc(&demo::App::reshape_cb);
541     glutDisplayFunc(&demo::App::display_cb);
542     glutMouseFunc(&demo::App::mouse_cb);
543     glutMotionFunc(&demo::App::motion_cb);
544     glutKeyboardFunc(&demo::App::keyboard_cb);
545     glutSpecialFunc(&demo::App::special_cb);
546
547     glutMainLoop();
548 }
```